Chapter 4: Interprocess Communication (IPC) & Synchronization

- Interptocess communication is a mechanism allows processes to communicate each other and synchronized action .
- Process is defined as a serie of action

Types of Interprecess communication

- Processes executing concurrently in the operating systems may be either independent processes or cooperating processes

- Independent processes – they can not be affected by the other processes executing in the system.
- Cooperating processes – they can be affected by other processes executing in the system..

- Any process that shares data with other process is a cooperating process

Reasons why or Advantages that allows process cooperation

- **Information sharing** /process need to share tasks like file , printer ,scanner ….
- **Resources sharing :** like harware divices , printer ,scanner , files …
- **Computation speedup**
- **Synchronization**
- **Modularity** / the task can be subvided into small tasks or modules to speed up the process
- **Convenience**

Cooperating processes require and interprocess communication (IPC) mechanism that will allow them to exchange data and information

Why do we need IPC?

- Each process operates sequentially
- All is fine until processes want to share data
  - Exchange data between multiple processes
  - Allow processes to navigate *critical regions*
  - Maintain proper sequencing of actions in multiple processes
- These issues apply to threads as well
  - Threads can share data easily (same address space)
  - Other two issues apply to threads

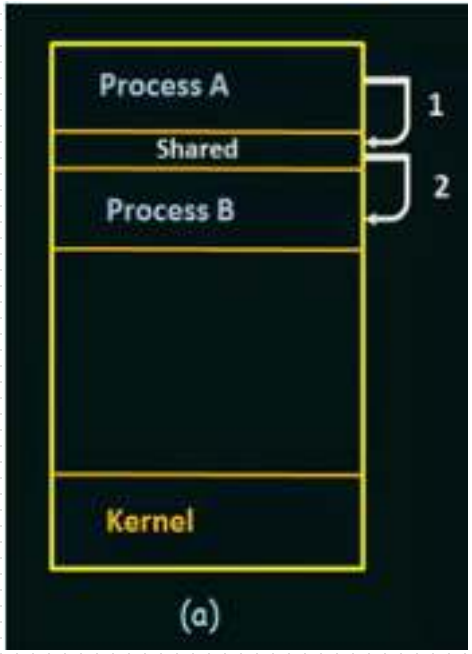
IPC mechanisms

Process can communicate each other by using two fundamental models of interprocess communication

- Memory sharing
- Message passing

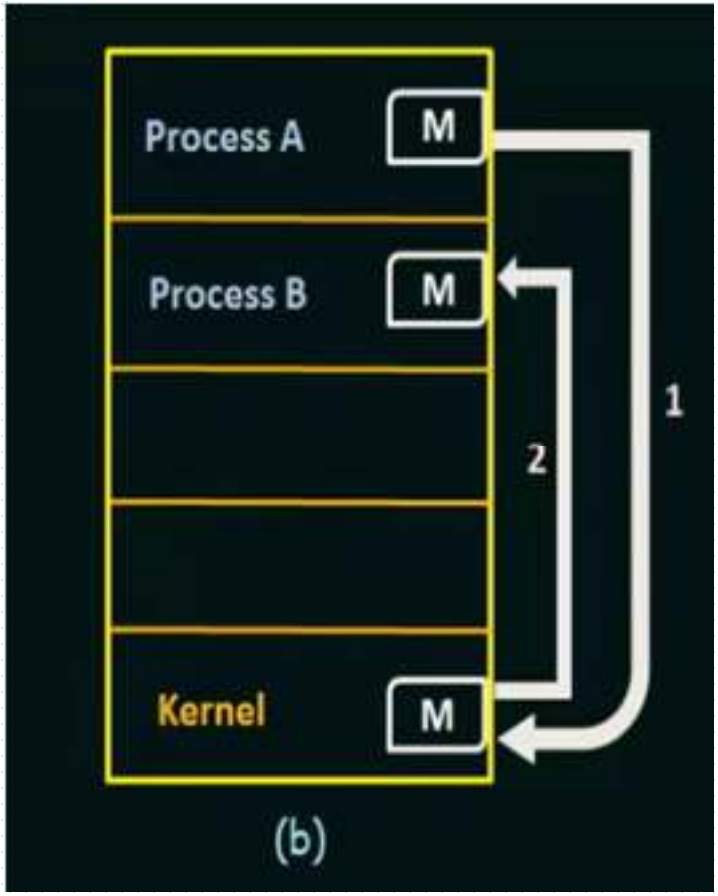
Memory sharing model

- A region of memory that shared by cooperating processes is established
- Processes can then excahnge information by reading and writing data to the shared region



Message Passing

- Message passing provides a mechanism to allow processes to communicate and synschronize their actions without sharing same address space.
- It is particulary useful in a distributed environment , where the communicating processes may reside on different computers connected by a network.
- A message-passing facility provided
- How?



- P 1 and P2 want to commincate each other

1. It has to establish a communication link
2. Start exchange message using basic primitives: send () and receive ()

- Naming

  -- Direct Communication -----> send (Q, message) and receiver (P, message)

  we have to specify the processing

  --- Indirect Communication (Processes communicate immediately each other P → mail box /port → Q

- Synchronization

  -- Blocked sender /Blocked receiver ----> synchonous

  -- Non Blocked sender /Non Blocked receiver --> Assynchronous

- Buffering the size of buffer can be checked before to check if the processes can be processed

  --- o capacity (zero capacity

  --- Finite

  --- Infinite

Basic Synchronization Principles

Synchronization is the coordination of the activities of two or more processes that usually need to carry out the following activities:

- Compete for resources in a mutually exclusive manner.
- Cooperate in sequencing or ordering specific events in their individual activities.

Example: bounded buffer problem

PRODUCER:

```
While(Items_number ==buffer size)
      ;   //waiting since the buffer is full

    Buffer[i]=next_produced_item;
     i=(i+1)%Buffer_size;
    Items_number++;
```

CONSUMER:

```
while (Items_number == 0)
                        ; // do nothing
since the buffer is empty

                Consumed _item=
buffer[j];
                              j = (j + 1)
% Buffer_size;
                        Items_number--;
```

bounded buffer problem

- One counter counts as we add data and stops a producer if there are N objects in the buffer.
- A second counter counts as we remove data and stops a consumer if there are 0 in the buffer.
- This means that :

  - The producer tries to insert data into an empty slots of the buffer.
  - The consumer tries to remove data from filled slots in the buffer.

- The producer must not insert the data when the buffer is full.
- The consumer must not remove data when the buffer is empty
- The producer and the consumer should not insert or remove data simultaneously.

Solution to bounded buffer problem using semaphore

- We will use of three semaphores :
- Mutex (m) : a binary which is used to acquire and release the lock
- Empyt : a counting semaphore whose initial value is the number of slots in the buffer, since initially all slots are empty
- Full : a counting semaphore whose initial value is 0

Producer case :

```
do {
  wait (empty); // wait until empty is > 0 and decrement empty
  wait(mutex);  // acquire

  /* add data to buffer */
  signal(mutex);  // release the lock
  signal (full); // increment full
  }
  while (TRUE)
```

Consumer case

```
do {
  wait (full); // wait until full is > 0 and decrement full
  wait(mutex);  // acquire

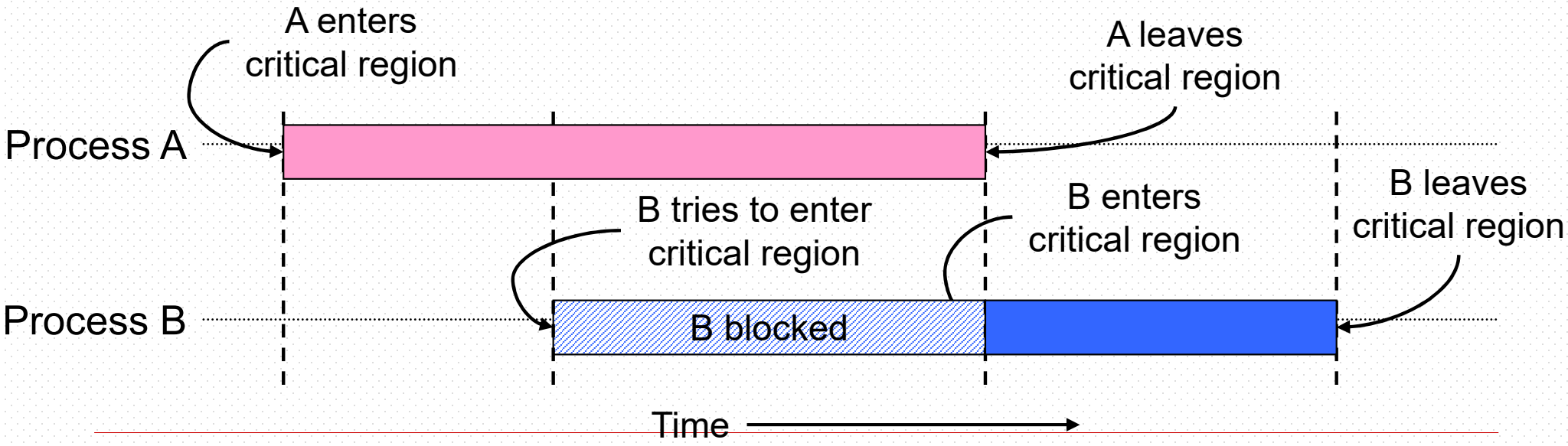
  /* add data to buffer */
  signal(mutex);  // release the lock
  signal (empty); // increment empty
  }
  while (TRUE)
```

Problem: race conditions

- Cooperating processes share storage (memory)
- Both may read and write the shared memory
- Problem: can't guarantee that read followed by write is atomic
  - Ordering matters!
- This can result in erroneous results!
- We need to eliminate race conditions…

Critical regions:

- Use critical regions to provide *mutual exclusion* and help fix race conditions
- Four conditions to provide mutual exclusion
  - No two processes simultaneously in critical region
  - No assumptions made about speeds or numbers of CPUs
  - No process running outside its critical region may block another process
  - No process must wait forever to enter its critical region



Mutual Exclusion

- Mutual Exclusion is the way of making sure that if one process is executing critical section, other processes will be excluded from executing critical section
- It is the responsibility of operating system to assure that no more than one process is in its critical section simultaneously.

Strict Alteration

- The integer variable turn, initially 0, keeps track of whose turn it is to enter the critical region and examine or update the shared memory.
- Initially, process 0 inspects turn, finds it to be 0, and enters its critical region.
- Process 1 also finds it to be 0 and therefore sits in a tight loop continually testing turn to see when it becomes 1.
- Continuously testing a variable until some value appears is called **busy waiting.**

Problem with Strict Alteration

- Now consider the following situation for two processes P0 and P1.
  - ✔ Suddenly, process 0 finishes its noncritical region and goes back to the top of its loop.
  - ✔ Unfortunately, it is not permitted to enter its critical region now, because turn is 1 and process 1 is busy with its noncritical region.
  - ✔ It hangs in its while loop until process 1 sets turn to 0.
  - ✔ See it differently, taking turns is not a good idea when one of the processes is much slower than the other.

Busy waiting: This is a process whereby the CPU spends most of its time in a tight loop waiting for a device to become ready

Process 0

```
while (TRUE) {
  while (turn != 0)
    ; /* loop */
  critical_region ();
  turn = 1;
  noncritical_region ();
}
```

Process 1

```
while (TRUE) {
  while (turn != 1)
    ; /* loop */
  critical_region ();
  turn = 0;
  noncritical_region ();
}
```

- Use a shared variable (turn) to keep track of whose turn it is
- Waiting process continually reads the variable to see if it can proceed
  - This is called a *spin lock* because the waiting process "spins" in a tight loop reading the variable
- Avoids race conditions, but doesn't satisfy criterion 3 for critical regions



Monitors

- A *monitor* is another kind of high-level synchronization primitive
  - One monitor has multiple entry points
  - Only one process may be in the monitor at any time
  - Enforces mutual exclusion - less chance for programming errors
- Monitors provided by high-level language
  - Variables belonging to monitor are protected from simultaneous access
  - Procedures in monitor are guaranteed to have mutual exclusion
- Monitor implementation
  - Language / compiler handles implementation
  - Can be implemented using semaphores

Monitor usage

```
monitor mon {
  int foo;
  int bar;
  double arr[100];
  void proc1(…) {
  }
  void proc2(…) {
  }
  void mon() {  // initialization code
  }
};
```

- This looks like C++ code, but it's not supported by C++
- Provides the following features:
  - Variables foo, bar, and arr are accessible only by proc1 & proc2
  - Only one process can be executing in either proc1 or proc2 at any time



Condition variables in monitors

- Problem: how can a process wait inside a monitor?
  - Can't simply sleep: there's no way for anyone else to enter
  - Solution: use a condition variable
- Condition variables support two operations
  - Wait(): suspend this process until signaled
  - Signal(): wake up exactly one process waiting on this condition variable
    - If no process is waiting, signal has no effect
    - Signals on condition variables aren't "saved up"
- Condition variables are only usable within monitors
  - Process must be in monitor to signal on a condition variable
  - Question: which process gets the monitor after Signal()?



Monitor semantics

- Problem: P signals on condition variable X, waking Q
  - Both can't be active in the monitor at the same time
  - Which one continues first?
- Mesa semantics
  - Signaling process (P) continues first
  - Q resumes when P leaves the monitor
  - Seems more logical: why suspend P when it signals?
- Hoare semantics
  - Awakened process (Q) continues first
  - P resumes when Q leaves the monitor
  - May be better: condition that Q wanted may no longer hold when P leaves the monitor



Locks & condition variables

- Monitors require native language support
- Provide monitor support using special data types and procedures
  - Locks (Acquire(), Release())
  - Condition variables (Wait(), Signal())
- Lock usage
  - Acquiring a lock == entering a monitor
  - Releasing a lock == leaving a monitor
- Condition variable usage
  - Each condition variable is associated with exactly one lock
  - Lock must be held to use condition variable
  - Waiting on a condition variable releases the lock implicitly
  - Returning from Wait() on a condition variable reacquires the lock

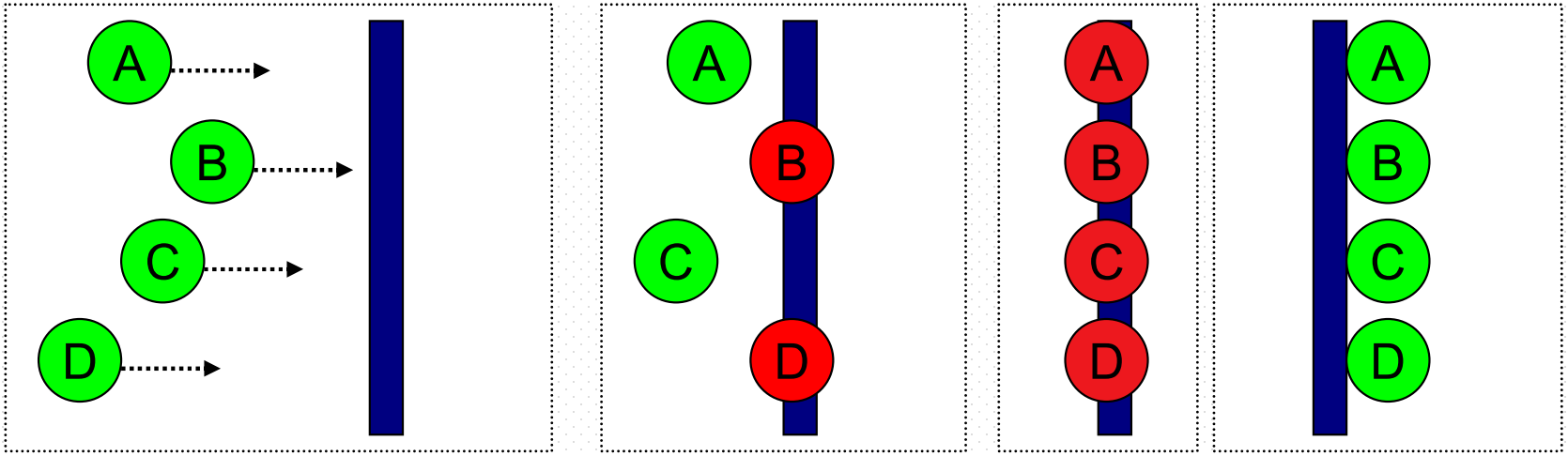


Message passing

- Synchronize by exchanging messages
- Two primitives:
  - Send: send a message
  - Receive: receive a message
  - Both may specify a “channel” to use
- Issue: how does the sender know the receiver got the message?
- Issue: authentication

Barriers

- Used for synchronizing multiple processes
- Processes wait at a “barrier” until all in the group arrive
- After all have arrived, all processes can proceed
- May be implemented using locks and condition variables



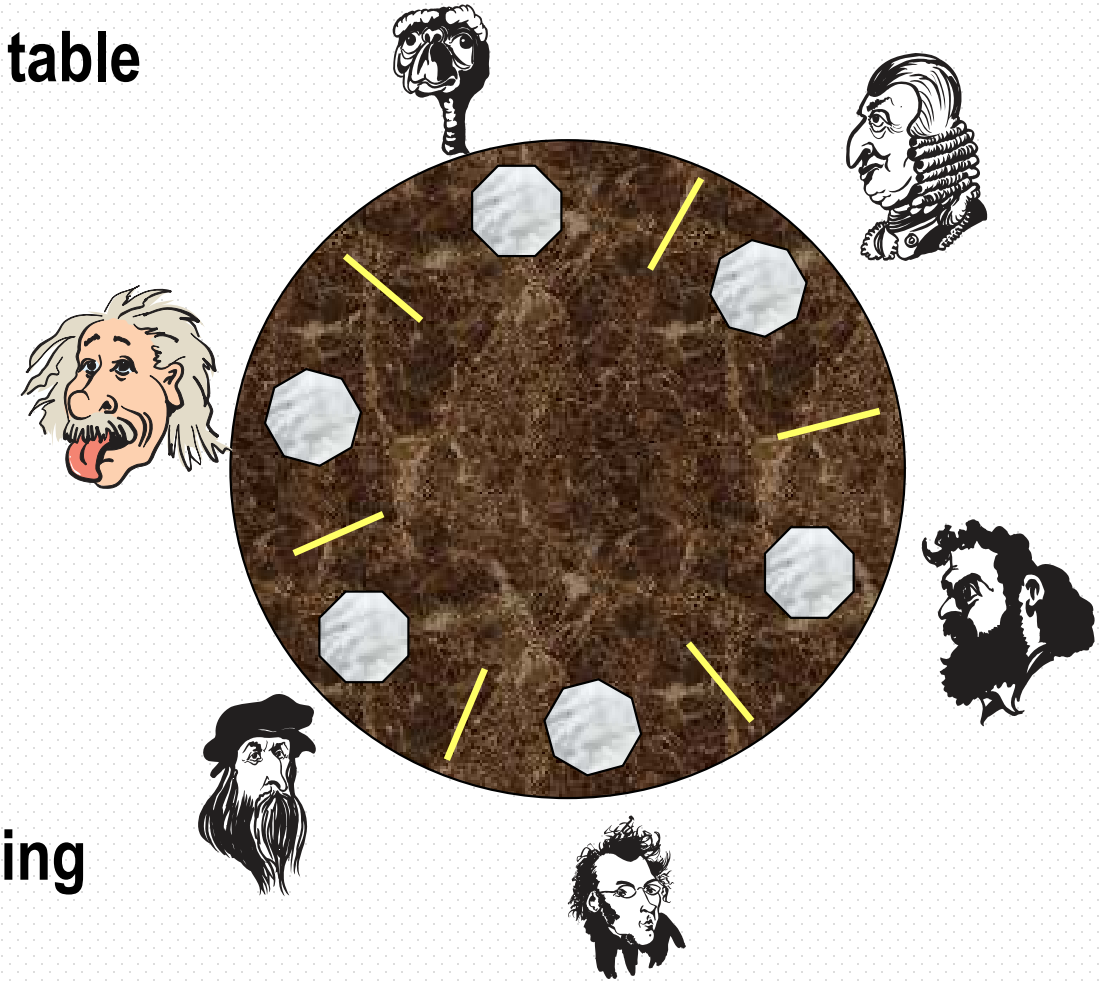
Processes approaching barrier | B and D at barrier | All at barrier | Barrier releases all processes

Deadlock and starvation

- Deadlock: two or more processes are waiting indefinitely for an event that can only by caused by a waiting process
  - P0 gets A, needs B
  - P1 gets B, needs A
  - Each process waiting for the other to signal
- Starvation: indefinite blocking
  - Process is never removed from the semaphore queue in which its suspended
  - May be caused by ordering in queues (priority)

Dining Philosophers Problem

- *N* philosophers around a table
  - All are hungry
  - All like to think
- *N* chopsticks available
  - 1 between each pair of philosophers
- Philosophers need two chopsticks to eat
- Philosophers alternate between eating and thinking
- Goal: coordinate use of chopsticks



Dining Philosophers: solution 2

- Use a semaphore for each chopstick
- A hungry philosopher
  - Gets lower, then higher numbered chopstick
  - Eats
  - Puts down the chopsticks
- Potential problems?
  - Deadlock
  - Fairness

Classical synchronization problems

- Bounded Buffer
  - Multiple producers and consumers
  - Synchronize access to shared buffer
- Readers & Writers
  - Many processes that may read and/or write
  - Only one writer allowed at any time
  - Many readers allowed, but not while a process is writing
- Dining Philosophers
  - Resource allocation problem
  - N processes and limited resources to perform sequence of tasks
- Goal: use semaphores to implement solutions to these problems



